

Face Recognition

Basic Theory

Face detection algorithms can be divided into two categories by method: **Feature-based algorithms** and **Image-based algorithms**.

- **Feature-based algorithms**

Feature-based algorithms extract features from images and match them with face features. If they match, it is considered a face; otherwise, it is not. The extracted features are manually designed features, such as Haar and HOG. After feature extraction, classifiers are used for judgment. Simply put, template matching is used, where the face template image is matched with each position in the image to be detected. The matched content is the extracted features, and then classifiers are used to determine whether there is a face.

- **Image-based algorithms**

Image-based algorithms divide the image into many small windows and determine whether each small window contains a face. Typically, image-based methods rely on statistical analysis and machine learning, finding statistical relationships between faces and non-faces through statistical analysis or learning processes for face detection. The most representative is CNN. **CNN used for face detection currently has the best effect and fastest speed.**

- **Haar Features**

We use **machine learning** methods to complete **face detection**. First, **we need a large number of positive sample images (face images) and negative sample images (images without faces) to train the classifier**. We need to **extract features from them**. The **Haar features** shown in the figure below will be used, similar to our convolution kernels. **Each feature is a value**, which equals the **sum of pixel values in the black rectangle** minus the **sum of pixel values in the white rectangle**.

This tutorial uses Haar cascade classifiers, which is the most common and efficient object detection method. This machine learning method is based on training cascade functions with a large number of positive and negative images, then used to detect objects in other images. If you don't want to create your own classifier, OpenCV also contains many pre-trained classifiers that can be used for face, eye, smile detection, etc.

Main Code

Code path: /home/yahboom/Dofbot/6.AI_Visual/6.Face recognition.ipynb

The following code content needs to be executed step by step according to actual needs. Do not run all at once. Running the last cell will directly exit the thread.

Import Haar cascade classifier xml file

```
self.faceDetect = cv.CascadeClassifier("haarcascade_frontalface_default.xml")
```

Face filtering

```

def face_filter(self, faces):
    '''
    Filter the face
    '''
    if len(faces) == 0: return None
    # At present, we are looking for the face with the largest area in the
picture
    max_face = max(faces, key=lambda face: face[2] * face[3])
    (x, y, w, h) = max_face
    # Set the minimum threshold of face detection
    if w < 10 or h < 10: return None
    return max_face

```

Recognition box selects face

```

def follow_function(self, img):
    img = cv.resize(img, (640, 480))
    # Copy the original image to avoid interference during processing
    # img = img.copy()
    # Convert image to grayscale
    gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
    # Face detection
    faces = self.faceDetect.detectMultiScale(gray, scaleFactor=1.3,
minNeighbors=5)

    if len(faces) != 0:
        face = self.face_filter(faces)
        # Face filtering
        (x, y, w, h) = face
        # Draw a rectangle on the original color map
        cv.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 4)
        cv.putText(img, 'Person', (280, 30), cv.FONT_HERSHEY_SIMPLEX, 0.8,
(105, 105, 105), 2)

```

Main thread:

```

def camera():
    global model
    # Open camera
    capture = cv.VideoCapture(0)
    while capture.isOpened():
        try:
            _, img = capture.read()
            img = cv.resize(img, (640, 480))
            img = follow.follow_function(img)
            if model == 'Exit':
                cv.destroyAllWindows()
                capture.release()
                break
            imgbox.value = cv.imencode('.jpg', img)[1].tobytes()
        except KeyboardInterrupt: capture.release()

```

After running the program, you can see faces detected and boxed.

